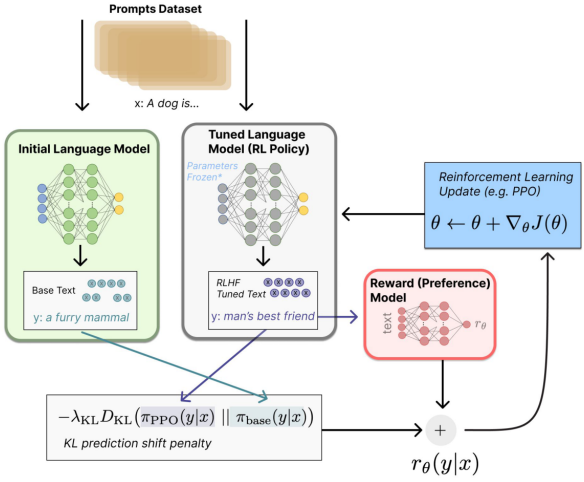
Fine-Tuning LLMs and RLHF

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 11, 2026



Lambert, Castricato, von Werra & Havrilla, "Illustrating Reinforcement Learning from Human Feedback (RLHF)," Hugging Face Blog, 2022.

Table of Contents

Table of Contents (cont.)

Table of Contents (cont.)

Motivation

- Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

Motivation

- Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.
- **We need:**
  - Adaptability (domain tuning)
  - Alignment (user intention)
  - Efficiency (cost-effective methods)

Motivating Examples:

▷ GPT-3 → InstructGPT

▷ LLaMA → Alpaca, Vicuna

▷ ChatGPT & Claude → RLHF-tuned

Motivation

- Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.
- **We need:**
  - Adaptability (domain tuning)
  - Alignment (user intention)
  - Efficiency (cost-effective methods)
- Fine-tuning and RLHF bridge the gap between general intelligence and practical usability.

Motivating Examples:

▷ GPT-3 → InstructGPT

▷ LLaMA → Alpaca, Vicuna

▷ ChatGPT & Claude → RLHF-tuned

Learning Outcomes

After this session, you will be able to:

- Explain and compare different fine-tuning methods for LLMs
- Understand the pipeline of Supervised Fine-Tuning and RLHF
- Describe PPO (Proximal Policy Optimization) and DPO (Direct Preference Optimization)
- Apply Low-Rank Adaptation (LoRA) and Quantized LoRA for efficient tuning
- Choose a PEFT method from task, data size, and serving constraints
- Explain model merging with task vectors, and when it beats retraining
- Compare ORPO and KTO against PPO and DPO for preference tuning
- Describe Constitutional AI and RLAIF, and the limits of AI feedback
- Analyze trade-offs, limitations, and future directions of LLM adaptation

Fine-Tuning Methods for LLMs

Categories of Fine-Tuning

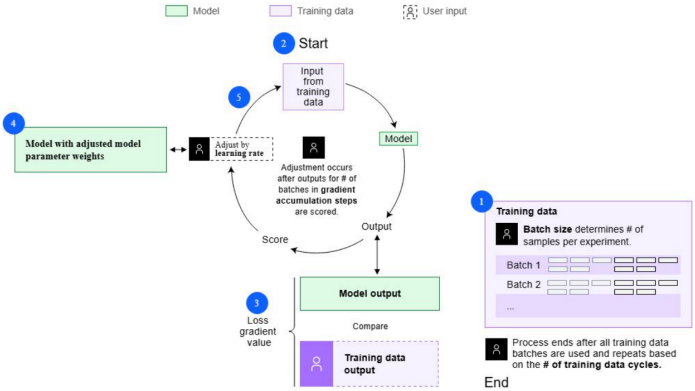
Method	Description	Trained params	Cost
Full FT	Retrain all weights	All	Expensive
Adapter	Insert small bottleneck layers (Houlsby et al., 2019)	Few	Efficient
Prefix / Prompt	Tune continuous prompt vectors, base model frozen	Few tokens	Efficient
LoRA / QLoRA	Low-rank weight deltas	Very few	Very efficient
Instruction tuning	Tune on instruction-response datasets	All or part	Varies

Instruction tuning is an orthogonal axis: it describes *what data* the model is tuned on, and can be done with any of the methods above.

Full Fine-Tuning (FT)

- **Definition:** Fine-tune all parameters of the pre-trained model on downstream data.
- **Historical Use:** Common in early GPT-2 and BERT applications.
- **Pros:**
  - Maximum flexibility and performance
- **Cons:**
  - Expensive (requires large compute resources)
  - Prone to catastrophic forgetting
  - Not efficient for large models

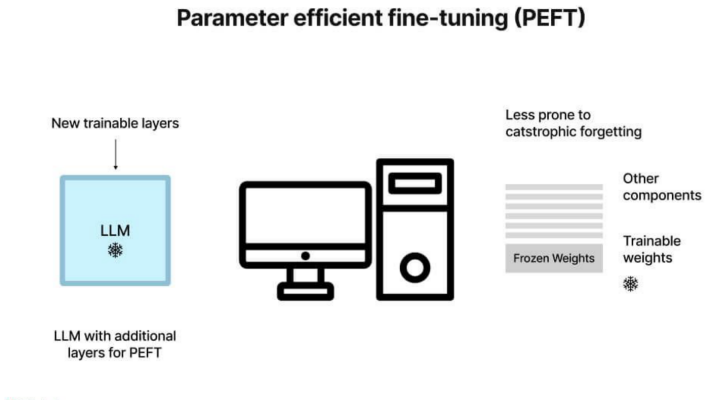
Full Fine-Tuning Workflow



Source: IBM watsonx.ai documentation, "Full fine tuning,"
https://www.ibm.com/docs/en/watsonx/w-and-w/2.1.0?topic=tuning-full-fine.

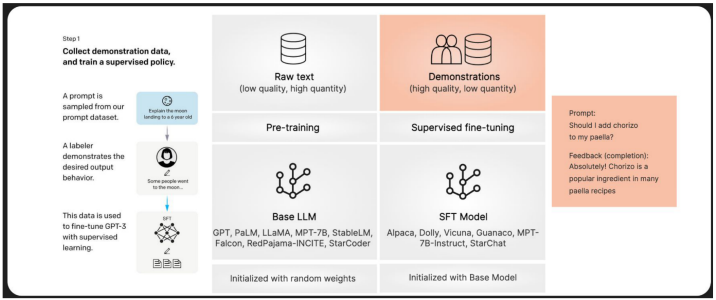
Parameter-Efficient Fine-Tuning (PEFT)

- **Key Idea:** Keep the base model frozen, tune only a small subset of parameters.
- **Types:**
  - Adapter Modules (Houlsby et al., 2019)
  - Prefix Tuning (Li & Liang, 2021) and Prompt Tuning (Lester et al., 2021)
  - LoRA / QLoRA
- **Benefits:**
  - Enables multi-tasking and personalization
  - Suitable for low-resource adaptation
  - Reduces compute and memory requirements


Parameter efficient fine-tuning (PEFT)
New trainable layers
LLM
LLM with additional layers for PEFT
Less prone to catstrophic forgetting
Other components
Frozen Weights
Trainable weights
SoluLab

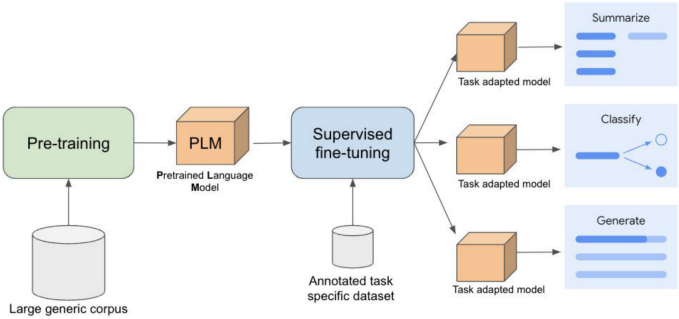
Supervised Fine-Tuning (SFT)

What is Supervised Fine-Tuning (SFT)?



Source: Argilla, "Collecting RLHF data" (docs.argilla.io); left panel from Ouyang et al., 2022.

Supervised Fine-Tuning For Gemini LLM



Supervised Fine Tuning for Gemini LLM

Source: E. Huizenga & M. Hu, "When to use supervised fine-tuning for Gemini," Google Cloud Blog, 2024.

Datasets for Supervised Fine-Tuning (SFT)

- **Instructional datasets:**
  - **OpenAI:** InstructGPT, roughly 13k labeler-written demonstrations (never released publicly)
  - **Stanford Alpaca:** 52k instructions generated with OpenAI's `text-davinci-003` using the Self-Instruct recipe
  - **Others:** databricks-dolly-15k, ShareGPT, OASST1, UltraChat
- Note where each one came from: Dolly and OASST1 are human-written, Alpaca and UltraChat are model-generated, ShareGPT is scraped conversations.
- **Note:** Quality of supervision greatly affects model behavior.

Domain-Specific SFT: Finance

- General instruction data teaches broad helpfulness; **domain-specific SFT** adapts a model to a vertical using labeled in-domain data.
- **Finance datasets:**
  - **FiQA** (WWW'18 challenge): aspect-based financial sentiment and opinion-based question answering.
  - **Financial PhraseBank** (Malo et al., 2014): 4,845 financial-news sentences labeled positive / negative / neutral from an investor's point of view by annotators with a finance background.
  - **SEC filings (10-K):** long regulatory filings used to tune models that read earnings reports and risk statements.

Two Routes to a Domain Model

- **Fine-tune an existing model. FinBERT** (Araci, 2019) is this recipe exactly: BERT further trained on a Reuters financial corpus, then fine-tuned on Financial PhraseBank. It reaches 0.86 accuracy where a same-size general model reaches 0.83.
- **Pre-train from scratch on a mixed corpus. BloombergGPT** used 363B financial and 345B general tokens. Far more expensive, and worth it only with proprietary data at that scale.
- For almost everyone the first route is the right one, and the same recipe applies to law, medicine, and code: a modest, high-quality in-domain set often beats a much larger generic one.

Limitations of Supervised Fine-Tuning (SFT)

- Cannot capture nuanced human preferences or values.
- May reinforce existing biases or hallucinations present in the data.
- Risk of overfitting, especially on synthetic or noisy datasets.
- Often leads to safe but bland and generic responses.

RLHF — Reinforcement Learning with Human Feedback

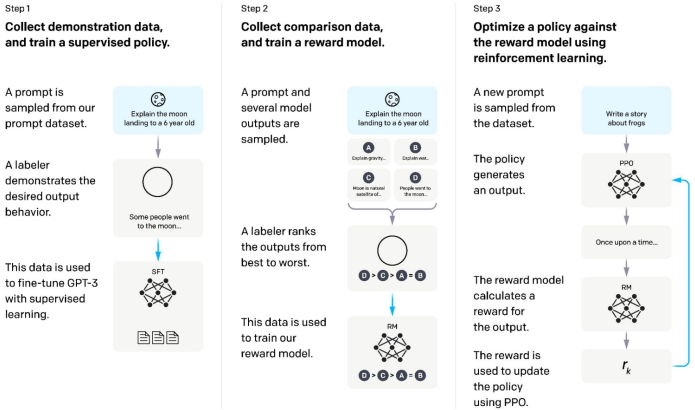
Why RLHF?

- SFT doesn't teach the model what humans prefer.
- RLHF introduces reward learning and policy optimization.
- Inspired by InstructGPT (Ouyang et al., 2022).

RLHF Pipeline Overview

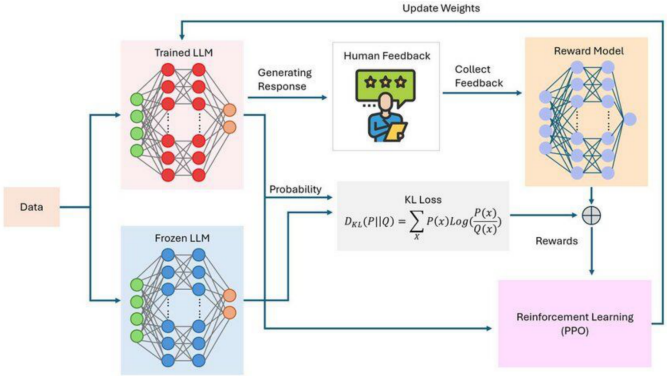
1. **Supervised Fine-Tuning (SFT):** Train the base model on instruction-response pairs.
2. **Reward Model (RM) Training:** Learn a reward function from human preference rankings.
3. **RL Optimization (e.g., PPO):** Optimize the language model to maximize the learned reward.

RLHF Pipeline



Ouyang et al., "Training language models to follow instructions with human feedback," NeurIPS 2022, Fig. 2.

RLHF Pipeline



Source: Wu, You & Du, "AI generations: from AI 1.0 to AI 4.0," *Frontiers in Artificial Intelligence* 8:1585629, 2025, Fig. 7.

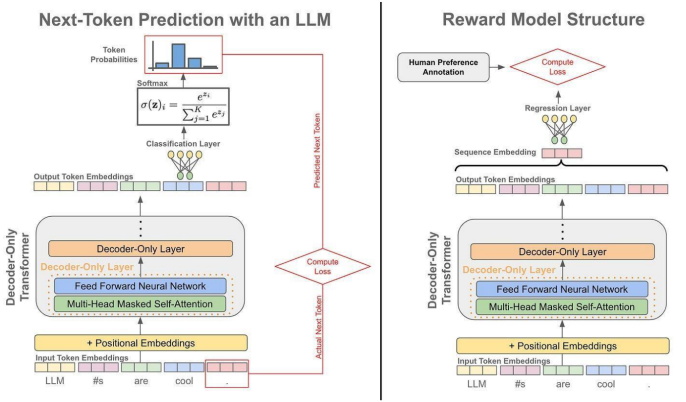
Reward Model (RM)

- **Given:** A prompt x and two responses to it. The annotator marks one as preferred, giving a winner y_w and a loser y_l.
- **Architecture:** Start from the SFT model and replace its next-token output layer with a scalar head, so $r_\phi(x, y)$ returns one score for a prompt-response pair.
- **Pairwise loss** (the Bradley–Terry preference model):

$$L_{\mathrm{RM}} = -\log \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big)$$

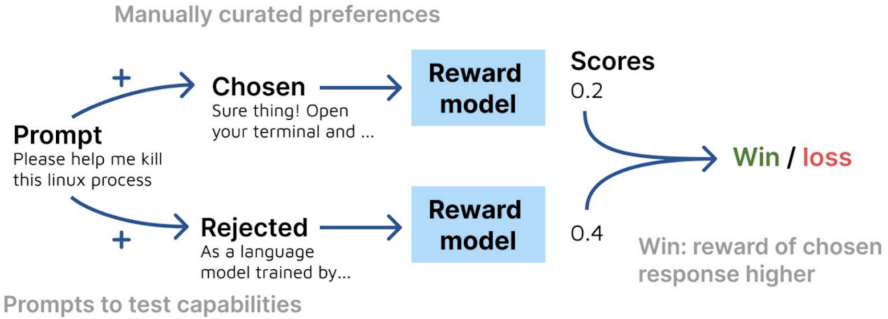
  where σ is the sigmoid function.
- The loss constrains only the *gap* between the two scores, so the absolute reward scale is arbitrary.
- The RM acts as a proxy for human judgment in downstream RL optimization.

Reward Model Architecture



Source: Cameron R. Wolfe, "The Story of RLHF," Deep (Learning) Focus, 2023.

Scoring a Preference Pair



Source: Lambert et al., "RewardBench: Evaluating Reward Models for Language Modeling," 2024, Fig. 1.

PPO in the LLM Setting

Text generation is cast as a sequential decision problem, one token at a time:

- **Policy π_θ:** the language model itself; $\pi_\theta(a_t \mid s_t)$ is its next-token distribution.
- **State s_t:** the prompt plus every token generated so far.
- **Action a_t:** the next token, drawn from the vocabulary.
- **Rollout:** one complete sampled response to a prompt.
- **Reward:** the RM scores the *finished* response, so one number has to be credited back across every token that produced it.
- **Advantage $\hat{A}_t$:** how much better action a_t turned out than expected at s_t. A separate value model, trained alongside the policy, supplies that expectation.

RLHF Training: Proximal Policy Optimization (PPO)

- **Objective:** Maximize the reward given by the reward model (RM).
- **Reward with KL penalty:** the RM score is shaped so the policy cannot drift far from the SFT model:

$$R(x, y) = r_\phi(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{SFT}}(y \mid x)}$$

- **Clipped surrogate objective**, which PPO *maximises*:

$$L_{\text{PPO}} = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta)\hat{A}_t, \quad \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t \right) \right]$$

  where $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ is the probability ratio (not a reward) and $\hat{A}_t$ is the advantage estimate.

- Clipping caps how far a single update can move the policy, which is what keeps the optimization stable.

PPO — Pros and Cons

- ✓ Stable optimization: the clip bounds every policy update.
- ✓ Learns from the model's own samples, so it can improve on behaviour never demonstrated in the SFT data.
- × Expensive: each step needs fresh rollouts, and four networks are in play at once, the policy, a frozen SFT copy for the KL term, the reward model, and the value model.
- × Sensitive to reward model errors: the policy optimizes the RM, not the humans behind it.
- × Many interacting hyperparameters (β, ϵ, rollout size, value-loss weight).

Direct Preference Optimization (DPO)

- **Key idea:** Optimize the policy directly on preference pairs, with no reward model and no RL rollouts.
- **Reference:** Rafailov et al., 2023.

DPO loss, on the same preference triples (x, y_w, y_l) used to train an RM:

$$L_{\text{DPO}} = -\log \frac{\exp\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}\right)}{\exp\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}\right) + \exp\left(\beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)}$$

where y_w is the preferred response, y_l the dispreferred one, π_θ the policy being trained, π_{ref} the frozen reference (SFT) policy, and β controls how far π_θ is allowed to move away from π_{ref}.

Why DPO Works

- Since $\frac{e^a}{e^a+e^b} = \sigma(a-b)$, the loss above is the same expression written with a sigmoid:

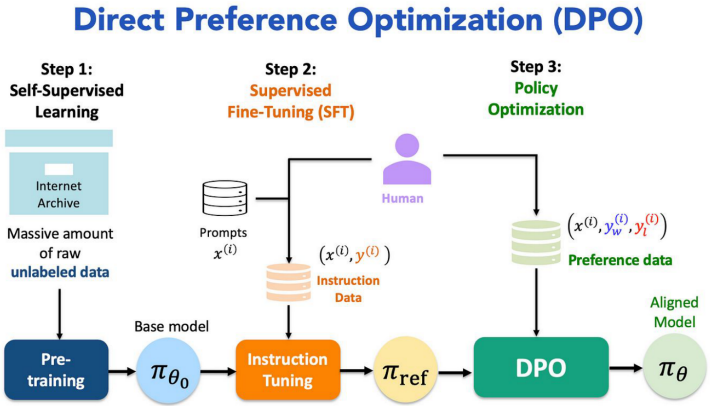
$$L_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

- Compare it with the reward-model loss $-\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$: the two are identical once we read $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ as an *implicit* reward.

- The policy is therefore its own reward model, which is what the paper's title refers to. Fitting the preferences fits the policy in one step.

Benefits:

- Simpler and more stable than PPO: a plain supervised loss on a fixed dataset.
- Two networks instead of four, and no sampling loop during training.



Source: L. M. Po, "Direct Preference Optimization (DPO) of LLMs: A Paradigm Shift," *Medium*, 2024.

Beyond DPO: ORPO and KTO

ORPO: One Stage, No Reference Model

- **Key idea:** DPO removed the reward model but kept two things: a separate SFT stage and a frozen reference policy. ORPO removes both (Hong et al., 2024).

Define the **odds** that the model produces y rather than not, and penalise the chosen response's odds being no better than the rejected one's:

$$\mathbf{odds}_\theta(y \mid x) = \frac{P_\theta(y \mid x)}{1 - P_\theta(y \mid x)}, \qquad \mathcal{L}_{\mathrm{OR}} = -\log \sigma\left(\log \frac{\mathbf{odds}_\theta(y_w \mid x)}{\mathbf{odds}_\theta(y_l \mid x)}\right)$$

This rides along with the ordinary SFT loss on the chosen response, in a single objective:

$$\mathcal{L}_{\mathrm{ORPO}} = \mathbb{E}_{(x,\, y_w,\, y_l)}\left[\, \mathcal{L}_{\mathrm{SFT}} + \lambda\, \mathcal{L}_{\mathrm{OR}} \,\right]$$

There is no π_{ref} anywhere in the expression: one model, one pass, one stage. The authors use λ between 0.1 and 0.25 depending on the base model.

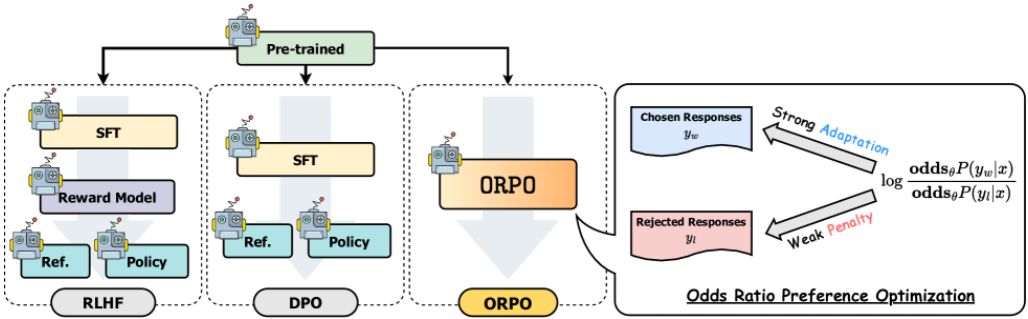
Why an Odds Ratio

- SFT has no mechanism for pushing anything *down*. It raises the likelihood of good answers and is indifferent to bad ones, which is why a second alignment stage was needed at all.
- The odds ratio supplies that missing downward pressure, but gently. Hong et al. compare it to the plain probability ratio and find the latter "leads to more extreme discrimination of the disfavored responses," suppressing token logits harder than a model still learning the task can absorb.
- The asymmetry is the whole design: *strong adaptation* toward the chosen response, *weak penalty* on the rejected one. That is what lets alignment share a stage with SFT instead of following it.
- **What it buys:** one model in memory instead of two or four, and one training run instead of two or three. Mistral-ORPO-β, trained on UltraFeedback alone, reports 12.20% on AlpacaEval 2.0 and 7.32 on MT-Bench.

Why an Odds Ratio (cont.)

- **What it costs:** the reference policy was also a leash. Without π_{ref} there is no KL term holding the model near its starting point, so λ and the learning rate carry more weight than they do in DPO.

ORPO Against RLHF and DPO



Hong, Lee & Thorne, "ORPO: Monolithic Preference Optimization without Reference Model," arXiv:2403.07691v2, Figure 2.

KTO: Thumbs Up and Thumbs Down

- **The data problem:** PPO, DPO and ORPO all need *pairs*: two responses to the same prompt, ranked. Production systems collect something else: one response, with a thumbs up or down.
- **KTO** (Ethayarajh et al., 2024) learns from that unpaired signal using the value function from Kahneman and Tversky's prospect theory, which is concave in gains, convex in losses, and measured against a reference point rather than in absolute terms:

$$r_\theta(x, y) = \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}, \qquad z_0 = \mathrm{KL}\big(\pi_\theta(y' \mid x) \,\|\, \pi_{\text{ref}}(y' \mid x)\big)$$

$$v(x, y) = \begin{cases} \lambda_D \, \sigma\big(\beta\,(r_\theta(x, y) - z_0)\big) & \text{if } y \text{ is desirable} \\ \lambda_U \, \sigma\big(\beta\,(z_0 - r_\theta(x, y))\big) & \text{if } y \text{ is undesirable} \end{cases} \qquad \mathcal{L}_{\text{KTO}} = \mathbb{E}\big[\lambda_y - v(x, y)\big]$$

z_0 is the model's KL from π_{ref}, estimated cheaply from mismatched pairs in the microbatch. λ_y is λ_D or λ_U as y requires; these loss-aversion weights keep $\lambda_D n_D / \lambda_U n_U$ in $[1, 3/2]$, so at a 1:10 desirable-to-undesirable ratio set $\lambda_U = 1$ and $\lambda_D \in [10, 15]$. r_θ still references π_{ref}: KTO drops the pairing, not the reference model.

Choosing a Preference Method

Method	Data it needs	Models	Stages	Main risk
PPO	Pairs, then prompts to roll out on	4	SFT, RM, RL	Instability; reward hacking
DPO	Preference pairs (x, y_w, y_l)	2	SFT, then DPO	Sensitive to the SFT starting point
ORPO	Preference pairs (x, y_w, y_l)	1	Single stage	No KL leash; λ matters more
KTO	Unpaired thumbs up / down	2	SFT, then KTO	Needs λ_D, λ_U set to the class balance

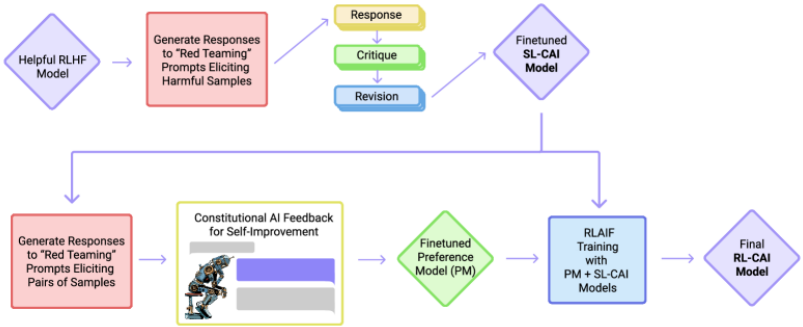
- **Start from the data you already have, not the leaderboard.** Unpaired feedback points at KTO, which tolerates skew paired methods cannot: discarding 90% of the desirable data still beat DPO on Llama-7B. Curated pairs on a tight budget point at ORPO; pairs plus an existing SFT model point at DPO.
- PPO stays the most expressive of the four, since it optimises against a reward model rather than a fixed dataset, and the hardest to get right.

Constitutional AI and RLAIF

Constitutional AI: Principles as Supervision

- **The problem with harmlessness data:** someone has to read the harmful output to label it. That does not scale, and it never says *why* something was rejected: the rule lives in the labelers' heads.
- **Constitutional AI** (Bai et al., 2022) replaces those labels with a written list of principles, and gets the model to apply them to itself, in two stages:
  - **SL-CAI (supervised).** Prompt a helpful-only model with red-team requests. Ask it to *critique* its own response against a randomly sampled principle, then *revise* it, and repeat. Fine-tune a pre-trained model on the revisions.
  - **RL-CAI (reinforcement).** Sample response pairs from the SL-CAI model, ask a feedback model which one the sampled principle prefers, and train a preference model on those labels: AI-written for harmlessness, still human for helpfulness. Then run RLHF against it.
- The RL stage used **16 principles**, one sampled at random per comparison, so no single wording dominates the preference model.
- The goal was a *harmless but non-evasive* assistant: one that engages with a harmful request and explains its objection instead of refusing flatly.

Constitutional AI: The Two Stages



Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073v1, Figure 1.

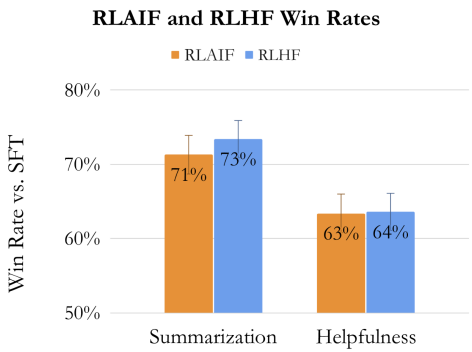
RLAIF: Replacing the Human Labeler

- **Key idea:** Everything in the RLHF pipeline stays the same except who produces the preference labels. An off-the-shelf LLM reads the prompt and two candidates and picks one (Lee et al., 2024).
- The labeling prompt has four parts: a preamble stating the task, optional few-shot exemplars, the pair to annotate, and an ending such as "`Preferred Response=`". Asking the labeler for its reasoning first (chain of thought) generally improves agreement with humans.
- **LLM labelers have position bias:** which candidate is shown first shifts the answer. The fix is mechanical: label each pair twice with the order reversed, and average the two preference distributions.
- **d-RLAIF** goes one step further and deletes the reward model: score each rollout with the off-the-shelf LLM directly during RL. This avoids a reward model that goes stale as the policy moves away from the data it was fit on.

RLAIF: Replacing the Human Labeler (cont.)

- **The economics:** human preference labels cost time and money per example; AI labels cost inference. The bottleneck moves from annotation throughput to having a labeler at least as capable as the model being aligned.

RLAIF Against RLHF



Lee et al., "RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback," arXiv:2309.00267v3, Figure 1, left panel. Human-judged win rates against the SFT baseline.

What AI Feedback Does and Does Not Buy

- **The headline result is parity.** Against an SFT baseline, human raters preferred RLAIF 71% of the time on summarisation and RLHF 73%; on helpful dialogue, 63% and 64%. Head to head the two are statistically indistinguishable: 50% on summarisation, 52% on dialogue.
- **On harmlessness the AI labeler was better:** 88% harmless, against 76% for RLHF and 64% for the SFT baseline. This is the setting where the written principle is easiest to state and hardest for a tired human to apply consistently.
- **It works with a same-size labeler.** On summarisation, an RLAIF run whose labeler was the same size as the policy still beat the SFT baseline 68% of the time, and d-RLAIF was preferred over that run 60/40. The gain is therefore not purely distillation from a larger model.

What AI Feedback Does and Does Not Buy (cont.)

- **The values come from the labeler.** You are copying a preference distribution that some other alignment process produced. Its blind spots become yours, silently, and no amount of scale surfaces them, which is why *evaluating* alignment is a separate discipline from producing it.
- **Looking ahead:** the same machinery now drives reasoning-model post-training, where the "labeler" is a verifier rather than a preference model. That story, and the RL algorithms built for it, belongs to a later session.

LoRA and Quantized LoRA

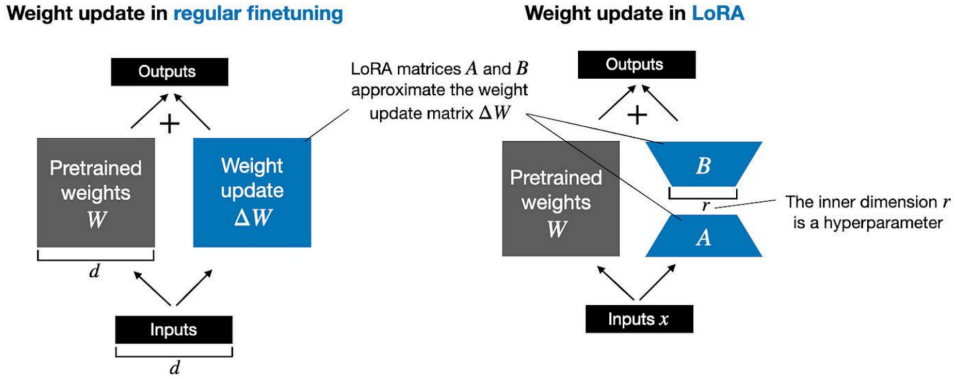
What is LoRA?

- **Key Idea:** Freeze the pre-trained weights and learn a low-rank update instead.
- For a weight matrix $W \in \mathbb{R}^{d \times k}$:

$$W' = W + \frac{\alpha}{r} \Delta W, \quad \Delta W = BA$$

  with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and rank $r \ll \min(d, k)$.
- Only A and B are trained: $r(d + k)$ parameters in place of dk.
- A is initialised from a Gaussian and B to zero, so $\Delta W = 0$ at the first step and training starts from exactly the pre-trained model.
- The scale α/r keeps the size of the update roughly independent of r, so changing the rank does not force a learning-rate retune.

What is LoRA?



Sebastian Raschka, "Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation)," *Ahead of AI*, 2023.

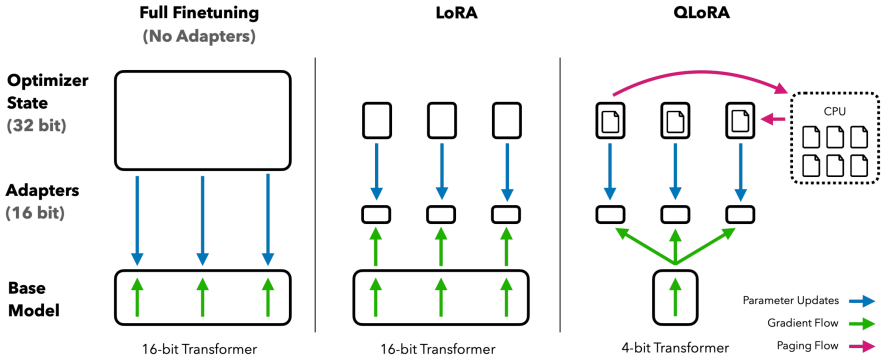
Benefits of LoRA

- **Very lightweight:** Well under 1% of parameters are trainable, and the fraction shrinks as the base model grows: about 0.2% for RoBERTa-large, and as little as 0.01% for GPT-3 175B, a 10,000× cut (Hu et al., 2021).
- **Hardware friendly:** Enables fine-tuning on consumer GPUs and even laptops.
- **No inference overhead:** BA can be folded back into W once training ends, so a deployed LoRA model runs at exactly the speed of the base model.
- **Modular:** Supports plug-and-play adapters for different tasks or users.
- **Personalization:** Allows efficient user- or domain-specific adaptation.

QLoRA — Quantized LoRA

- **Key Idea:** Fine-tune large language models in 4-bit precision using LoRA adapters.
- **Scalability:** Enables fine-tuning of 65B parameter models on a single 48GB GPU.
- **Efficiency:** Matches full 16-bit fine-tuning on the benchmarks tested; the authors note that parity at 33B and 65B was not established (Dettmers et al., 2023).
- **Techniques:**
  - 4-bit NormalFloat (NF4), a storage type matched to normally distributed weights
  - Double quantization: quantize the quantization constants too
  - Paged optimizers, to absorb memory spikes
- Weights are *stored* in 4 bits but dequantized to BFloat16 for each matrix multiply, so compute precision is unchanged.

QLoRA — Quantized LoRA



Dettmers, Pagnoni, Holtzman & Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," NeurIPS 2023, Fig. 1.

Applications of LoRA / QLoRA

- **Instruction tuning for resource-constrained settings:**
  - Enables fine-tuning large models on modest hardware (e.g., consumer GPUs, laptops).
  - Used for domain adaptation, personalization, and rapid prototyping.
- **Models tuned this way:**
  - **Guanaco** (Dettmers et al., 2023): a QLoRA fine-tune of LLaMA on OpenAssistant data; the 65B model trains in 24 hours on one 48GB GPU.
  - Base models such as **LLaMA** and **Mistral** are the *targets* of LoRA tuning, not products of it.
  - The original **Alpaca** and **Vicuna** releases were full fine-tunes on 8×A100; community LoRA reproductions came later.
- **Model merging and compositionality:**
  - Merge multiple LoRA adapters for multi-domain or multi-task capabilities.
  - Compose adapters for new tasks without retraining the base model.

PEFT Beyond LoRA

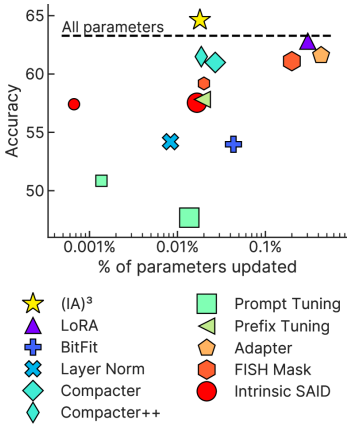
The PEFT Family at a Glance

Method	What is trained	Trainable	Folds into W?
Adapters	Bottleneck MLPs inserted in each block	$\sim$3.6%	No
Prefix tuning	Key/value prefixes at every layer	$\sim$0.1%	No
Prompt tuning	A soft prompt at the input layer only	$<$0.01%	No
LoRA	Low-rank update $\Delta W = BA$	0.01–0.2%	Yes
$(IA)^3$	Rescaling vectors on K, V, FFN	$\sim$0.02%	Yes
DoRA	Magnitude vector plus low-rank direction	$\sim$0.8%	Yes

Each percentage is for the base model that method's own paper used (BERT-base, GPT-2, T5-XXL, T0-3B, LLaMA-7B) and at that paper's own rank or bottleneck size, so the column is indicative, not a controlled comparison.

Folds into W is the serving question: LoRA, DoRA and $(IA)^3$ are absorbed into the frozen weights after training, so inference costs nothing extra. Adapters and prefixes cannot be.

Accuracy Against Parameter Budget



Liu et al., "Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning," arXiv:2205.05638v2, Figure 2. PEFT methods applied to T0-3B; the dashed line is full fine-tuning.

Soft Prompts: Prefix and Prompt Tuning

- **Key idea:** Leave every weight frozen and learn the *input* instead: a short sequence of continuous vectors that behave like tokens but never map to real words.
- **Prefix tuning** (Li & Liang, 2021) prepends trainable key/value vectors at *every* transformer layer, so later tokens attend to them as virtual tokens. About 0.1% of parameters. Optimising the prefix directly proved unstable, so it is reparameterised through a small MLP during training and discarded afterwards.
- **Prompt tuning** (Lester et al., 2021) simplifies this to the *input embedding layer only*, with no reparameterisation. A 100-token prompt for T5-XXL is 409,600 parameters, roughly 0.004% of the model.
- Neither method touches a single pre-trained weight, so one frozen copy of the model can serve every task you have ever tuned.

Soft Prompts: Where They Win

- **Storage.** A thousand tenants means a thousand prompt files of a few hundred kilobytes, on one shared model, rather than a thousand adapter checkpoints.
- **Small data.** Prefix tuning beats full fine-tuning at 50–500 training examples (+2.9 BLEU on average for table-to-text) and extrapolates better to unseen topics.
- **Domain shift.** Freezing the general-purpose weights limits overfitting: prompt tuning trained on SQuAD and evaluated zero-shot on TextbookQA beat full model tuning by 12.5 F1.

Soft Prompts: Where They Lag

- **Scale dependence.** Prompt tuning only reaches parity with full fine-tuning once the frozen model is large: Lester et al. show the gap closing at T5-XXL (11B) and widening steadily below it.
- **Context budget.** A 100-token prefix is 100 tokens the user no longer has, on every request forever.
- **No fold-in.** The prefix must be re-inserted at inference, so it is a permanent change to the serving path rather than a one-off weight edit.

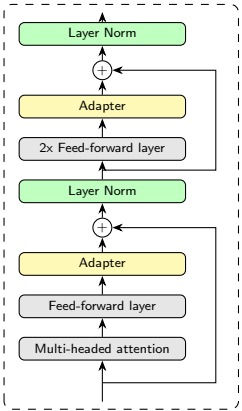
Adapters and $(IA)^3$

- **Adapters** (Houlsby et al., 2019) insert a bottleneck module twice per block: after the attention projection and after the feed-forward layers. Each is a $d \to m \to d$ bottleneck with a skip connection: $2md + d + m$ parameters with $m \ll d$, initialised near zero so the module starts as an approximate identity.
  - 3.6% of parameters per task, within 0.4% of full fine-tuning on GLUE, but real depth added to every forward pass.
- **$(IA)^3$** (Liu et al., 2022) adds no modules and trains no weights. It rescales three sets of *activations* element-wise with learned vectors l_k, l_v, l_{ff}:

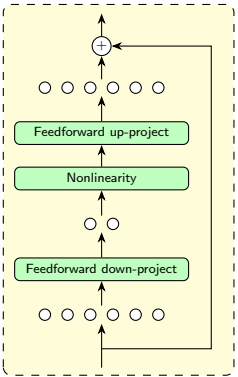
$$\text{softmax}\left(\frac{Q\,(l_k \odot K^\top)}{\sqrt{d_k}}\right)(l_v \odot V), \qquad \left(l_{\text{ff}} \odot \gamma(W_1 x)\right) W_2$$

540K parameters for T0-3B, up to 10,000$\times$ fewer than full fine-tuning. The two payoffs are alternatives: a single-task model can fold the vectors into the weights, since $l \odot Wx = (l \odot W)x$; kept separate, they let one batch serve several tasks at once.

Adapter Placement and Structure



Transformer layer with adapters



Adapter module

Redrawn after Houlsby et al., "Parameter-Efficient Transfer Learning for NLP," ICML 2019, arXiv:1902.00751, Figure 2. Green is trained, grey is frozen; each adapter sits after a sub-layer's projection and before the residual add.

DoRA: Magnitude and Direction

- **Observation:** split a weight matrix into how long each column is and where it points. Under full fine-tuning the two changes are negatively correlated (-0.62): columns that turn a lot do not also grow. LoRA ties them together ($+0.83$).
- **DoRA** (Liu et al., 2024) makes that split explicit before adapting:

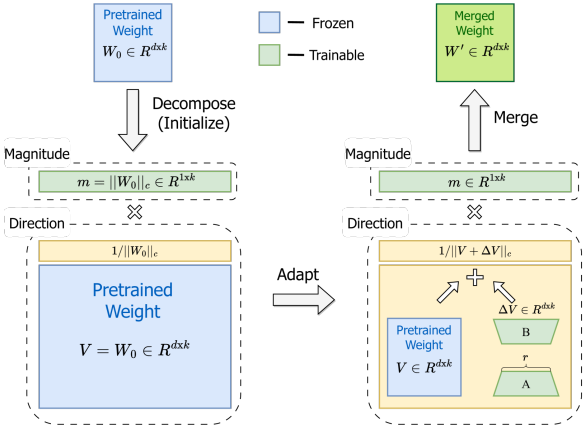
$$W = m \frac{V}{\|V\|_c}, \qquad m \in \mathbb{R}^{1\times k}, \quad V \in \mathbb{R}^{d\times k}$$

  where $\|\cdot\|_c$ is the vector norm taken down each column: m holds one magnitude per column, V holds the directions.
- Initialise $m = \|W_0\|_c$ and $V = W_0$, then adapt *only the direction* with LoRA while training the magnitude vector directly:

$$W' = m \frac{W_0 + BA}{\|W_0 + BA\|_c}$$

  m, B and A are trained; W_0 stays frozen, and the whole thing folds back into one matrix for serving.

DoRA: Magnitude and Direction



Liu et al., "DoRA: Weight-Decomposed Low-Rank Adaptation," arXiv:2402.09353v6, Figure 1.

Choosing a PEFT Method

- **Default to LoRA or DoRA.** Good accuracy, no inference overhead, and every serving stack supports them. The extra magnitude vector costs k numbers per matrix, so at matched rank the two train almost the same count (0.84% vs 0.83% on LLaMA-7B at $r = 32$), and DoRA holds its accuracy better as the rank falls: on commonsense reasoning it reaches 78.4 against LoRA's 74.7 on LLaMA-7B, and 85.2 against 80.8 on LLaMA3-8B.
- **Many tenants, one model?** Soft prompts or $(IA)^3$. Per-tenant state drops to kilobytes, and $(IA)^3$ additionally lets one batch serve several tenants.
- **Tens to hundreds of examples?** Soft prompts and $(IA)^3$ hold up better than methods with more capacity to overfit.
- **A genuinely new capability rather than a new style?** No PEFT method will get you there; that is a full fine-tune or continued pre-training question.
- **Adapters** are now mostly of historical interest for LLMs: they cost latency the fold-in methods do not, for no accuracy advantage.

Model Merging and Continual Learning

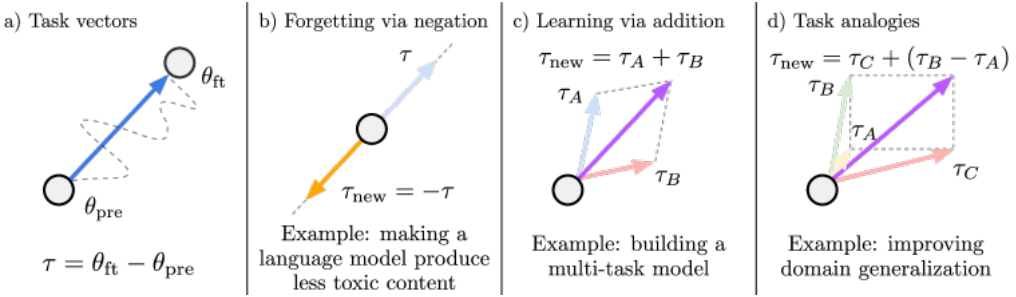
Task Vectors and Task Arithmetic

- **Key idea:** What a fine-tune *learned* is a direction in weight space. Subtract the starting point and you can keep it, move it, and add it to something else (Ilharco et al., 2023).

$$\tau = \theta_{\text{ft}} - \theta_{\text{pre}}, \qquad \theta_{\text{new}} = \theta_{\text{pre}} + \lambda \sum_i \tau_i$$

- This is element-wise arithmetic on checkpoints: no gradients and no retraining. The coefficient λ is the one real hyperparameter, and it is tuned on a held-out set, not derived.
- Three operations, all of which do something useful:
  - **Negation** ($-\tau$) degrades the target behaviour while leaving control tasks roughly intact: negating a toxicity vector took GPT-2 from 4.8% toxic generations to 0.8%.
  - **Addition** ($\tau_A + \tau_B$) builds a multi-task model without data from either task.
  - **Analogy** ($\tau_C + (\tau_B - \tau_A)$) improves a fourth task using only vectors from the other three.

Task Vectors and Task Arithmetic



Ilharco et al., "Editing Models with Task Arithmetic," ICLR 2023, arXiv:2212.04089v3, Figure 1.

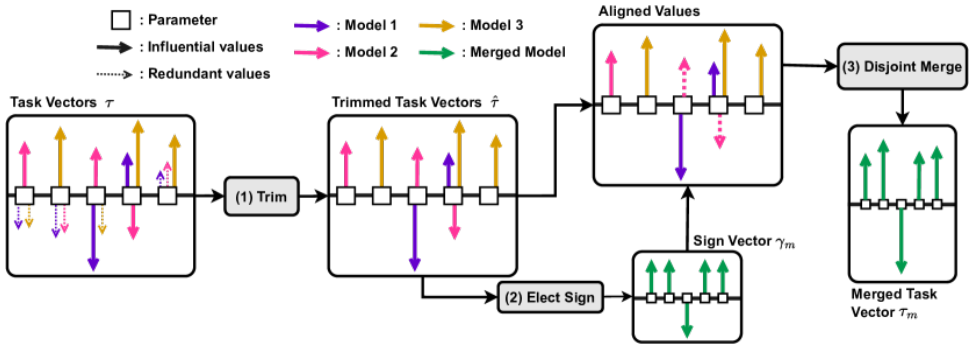
Merging Recipes

- **Linear (model soup):** a weighted average of the checkpoints. Cheap, and the baseline every other method is measured against.
- **SLERP:** interpolate along the geodesic rather than the chord, which avoids the drift toward the origin that straight-line averaging causes. Defined for two parents at a time.
- **TIES** (Yadav et al., 2023) attacks the two ways task vectors interfere (redundant entries, and sign disagreement) in three steps:
  - **Trim:** keep only the top 20% of each τ by magnitude, reset the rest to zero.
  - **Elect sign:** for each parameter, pick the sign whose total magnitude across models is larger.
  - **Disjoint merge:** average *only* the entries whose sign matches the elected one, leaving the zeros out of the denominator.

Merging Recipes (cont.)

- **DARE** (Yu et al., 2024) is a preprocessing step, not a merge: **d**rop delta parameters at rate p **a**nd **re**scale the survivors by $1/(1-p)$, so the expected update is unchanged. Dropping 90% of an SFT delta, and 99% on the largest models, costs little accuracy. It works only because those deltas are tiny; continued pre-training grows them and the method stops working. Note what is dropped: the *delta*, never the merged weight.
- **mergekit** (Goddard et al., 2024) implements all of these behind one config file, so trying a recipe costs minutes.

TIES-Merging: Trim, Elect Sign, Merge



Yadav, Tam, Choshen, Raffel & Bansal, "TIES-Merging: Resolving Interference When Merging Models," NeurIPS 2023, arXiv:2306.01708v2, Figure 1.

Merging LoRA Adapters, and When It Fails

- A LoRA adapter is *already* a task vector: $\tau = \frac{\alpha}{r} BA$. Fold each adapter into the base and merge the results, and every recipe above applies unchanged.
- **Do not average A and B separately.** The product is bilinear, so $\frac{1}{2}(B_1 + B_2)\frac{1}{2}(A_1 + A_2) \neq \frac{1}{2}(B_1A_1 + B_2A_2)$; the cross terms B_1A_2 and B_2A_1 are noise neither adapter ever trained. Merge in ΔW space, or concatenate the factors and let the rank grow.
- **Merging is only defined between homologous models:** the same architecture from the same pre-trained checkpoint. Two models with the same shape but different pre-training have no shared coordinate system, and the average is nonsense.
- **The failure mode is silent.** A bad merge still loads, still generates fluent text, and still scores respectably on the benchmarks the parents were good at. Safety behaviour and instruction-following degrade first and are exactly what a quick eval will not catch. Evaluate the merge, never the recipe.

Continual Learning for Deployed Models

- A deployed model is never finished: new domains, new products, new policy. Retraining from scratch each time is unaffordable, and tuning on only the new data causes **catastrophic forgetting**: the update overwrites weights the old capabilities depended on.
- **Replay.** Mix a slice of the original training data back into every new run. The simplest defence, and usually the most effective one.
- **Regularisation.** Penalise movement in the directions that mattered before. *Elastic weight consolidation* (Kirkpatrick et al., 2017) weights that penalty per parameter by the Fisher information of the old task.
- **Constrain the update.** Biderman et al. (2024) found LoRA learns *less* than full fine-tuning on code, and by a smaller margin on maths, because full fine-tuning finds updates of 10 to 100 times the rank. It also forgets less, holding the base model's behaviour outside the target domain better than weight decay or dropout do.

Continual Learning for Deployed Models (cont.)

- **Merge instead of retrain.** Keep the deployed model, train the new capability separately, and merge. You get a rollback path: the parents are still on disk.

Limitations of Fine-Tuning and RLHF

- **Data Quality:** RLHF relies heavily on high-quality, diverse preference data or human feedback; poor data leads to suboptimal or biased models.
- **Scalability:** Collecting human feedback at scale is expensive and time-consuming.
- **Alignment Gaps:** RLHF may not perfectly capture complex, nuanced human values or intent.
- **Overoptimization:** Models may "game" the reward model, leading to reward hacking or unnatural outputs.
- **Generalization:** Fine-tuning and RLHF can cause models to overfit to specific tasks or feedback distributions, reducing robustness.

Limitations of Fine-Tuning and RLHF (cont.)

- **Catastrophic Forgetting:** Risk of losing previously learned capabilities during aggressive fine-tuning.
- **Computational Cost:** Both fine-tuning and RLHF are resource-intensive, especially for large models.
- **Ethical Concerns:** Human preferences used for training may encode societal biases, under-representation, or other ethical issues.
- **Transparency:** RLHF pipelines can be hard to interpret and debug.

Future Directions

- **Open RLHF Datasets:** Promote better community sharing and reproducibility with open preference datasets.
- **Multi-modal Alignment:** Extend alignment techniques to vision, audio, and code generation tasks.
- **Reward Hacking Defense:** Build reward models that hold up under heavy optimization, since gold-standard quality starts to fall once the proxy reward is pushed too far (Gao et al., 2023).

Summary

- Fine-tuning enables specialization and alignment of LLMs.
- Supervised Fine-Tuning (SFT) is the foundational step.
- RLHF incorporates human preference alignment using PPO.
- DPO streamlines the process with a direct preference loss.
- ORPO and KTO go further: one stage and no reference model, or unpaired thumbs up and down in place of ranked pairs.
- Constitutional AI and RLAIF move the labeling onto a model, trading annotation cost for values inherited from that labeler.
- LoRA and QLoRA make adaptation efficient and accessible; soft prompts, $(IA)^3$ and DoRA trade accuracy against serving cost differently.
- Task vectors combine finished fine-tunes by arithmetic, and are the cheapest answer to a model that has to keep changing.

Summary (cont.)

- Which method to reach for is largely a budget question: SFT to add a capability, preference tuning to shape behaviour, LoRA or QLoRA when memory is the binding constraint.

References

1. Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback (InstructGPT). https://arxiv.org/abs/2203.02155
2. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. https://arxiv.org/abs/1707.06347
3. Rafailov, R., Sharma, A., Mitchell, E., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. https://arxiv.org/abs/2305.18290
4. Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. https://arxiv.org/abs/2204.05862
5. Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. https://arxiv.org/abs/2212.08073
6. Gao, L., Schulman, J., & Hilton, J. (2023). Scaling Laws for Reward Model Overoptimization. ICML 2023. https://arxiv.org/abs/2210.10760
7. Lambert, N., Castricato, L., von Werra, L., & Havrilla, A. (2022). Illustrating Reinforcement Learning from Human Feedback (RLHF). Hugging Face Blog. https://huggingface.co/blog/rlhf
8. Lambert, N., Pyatkin, V., Morrison, J., et al. (2024). RewardBench: Evaluating Reward Models for Language Modeling. https://arxiv.org/abs/2403.13787

References (cont.)

9. Houlsby, N., Giurgiu, A., Jastrzebski, S., et al. (2019). Parameter-Efficient Transfer Learning for NLP. ICML 2019. https://arxiv.org/abs/1902.00751

10. Li, X. L., & Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL-IJCNLP 2021. https://arxiv.org/abs/2101.00190

11. Lester, B., Al-Rfou, R., & Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP 2021. https://arxiv.org/abs/2104.08691

12. Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685

13. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. https://arxiv.org/abs/2305.14314

14. Taori, R., Gulrajani, I., Zhang, T., et al. (2023). Stanford Alpaca: An Instruction-following LLaMA Model. Stanford CRFM. https://crfm.stanford.edu/2023/03/13/alpaca.html

15. Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. https://arxiv.org/abs/1908.10063

16. Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. *JASIST* 65(4), 782–796.

17. Wu, S., Irsoy, O., Lu, S., et al. (2023). BloombergGPT: A Large Language Model for Finance. https://arxiv.org/abs/2303.17564

References (cont.)

18. Liu, H., Tam, D., Muqeeth, M., et al. (2022). Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning ($(IA)^3$ / T-Few). NeurIPS 2022. https://arxiv.org/abs/2205.05638

19. Liu, S.-Y., Wang, C.-Y., Yin, H., et al. (2024). DoRA: Weight-Decomposed Low-Rank Adaptation. ICML 2024. https://arxiv.org/abs/2402.09353

20. Biderman, D., Portes, J., Gonzalez Ortiz, J. J., et al. (2024). LoRA Learns Less and Forgets Less. https://arxiv.org/abs/2405.09673

21. Ilharco, G., Ribeiro, M. T., Wortsman, M., et al. (2023). Editing Models with Task Arithmetic. ICLR 2023. https://arxiv.org/abs/2212.04089

22. Wortsman, M., Ilharco, G., Gadre, S. Y., et al. (2022). Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time. ICML 2022. https://arxiv.org/abs/2203.05482

23. Shoemake, K. (1985). Animating Rotation with Quaternion Curves (SLERP). *SIGGRAPH Computer Graphics* 19(3), 245–254.

24. Yadav, P., Tam, D., Choshen, L., Raffel, C., & Bansal, M. (2023). TIES-Merging: Resolving Interference When Merging Models. NeurIPS 2023. https://arxiv.org/abs/2306.01708

25. Yu, L., Yu, B., Yu, H., Huang, F., & Li, Y. (2024). Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch (DARE). ICML 2024. https://arxiv.org/abs/2311.03099

26. Goddard, C., Siriwardhana, S., Ehghaghi, M., et al. (2024). Arcee's MergeKit: A Toolkit for Merging Large Language Models. https://arxiv.org/abs/2403.13257

References (cont.) (cont.)

27. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. (2017). Overcoming catastrophic forgetting in neural networks. *PNAS* 114(13), 3521–3526. https://arxiv.org/abs/1612.00796

28. Hong, J., Lee, N., & Thorne, J. (2024). ORPO: Monolithic Preference Optimization without Reference Model. https://arxiv.org/abs/2403.07691

29. Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., & Kiela, D. (2024). KTO: Model Alignment as Prospect Theoretic Optimization. https://arxiv.org/abs/2402.01306

30. Lee, H., Phatale, S., Mansoor, H., et al. (2024). RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback. ICML 2024. https://arxiv.org/abs/2309.00267